

Data Quality Agent: An End-to-End Machine Learning System for Automated Data Quality Detection

Technical Report

Author: Evan Middleton

September 11, 2026

Executive Summary

The Data Quality Agent is an end-to-end binary classification system developed to identify potentially low-quality records before they reach analysts and business users. It uses a 50,000-record synthetic dataset with seven predictive attributes and a binary needs_review target. Only 10.22% of records required review, making class imbalance a central challenge.

Four Logistic Regression configurations were evaluated. The baseline achieved 90.23% accuracy but only 10.18% recall. Class weighting increased recall to 66.34% but reduced precision to 21.63%. The final approach used a balanced Logistic Regression model, a 0.70 decision threshold, and hyperparameter tuning. On the untouched test set it achieved 86.16% accuracy, 33.95% precision, 37.48% recall, and 35.63% F1.

The final model was registered and successfully deployed through Azure Machine Learning. A live endpoint test produced a 0.993 review probability and a REVIEW REQUIRED decision for a high-risk sample. The project demonstrates a complete machine learning workflow while identifying limitations that must be addressed before production use.

1. Introduction

Organizations rely on large datasets for analysis, reporting, automation, and decision-making, but data quality problems can reduce the value of those datasets. Missing values, duplicates, formatting errors, unusual response characteristics, low review scores, and elevated processing times can increase manual review effort and introduce risk into downstream analytics.

The Data Quality Agent addresses this problem by predicting whether an individual record should be sent for manual review. Instead of requiring analysts to inspect every record equally, the system produces a probability and a review decision that can be used to prioritize attention.

The objectives were to prepare the data, establish a baseline, compare modeling strategies, address class imbalance, optimize the classification decision, evaluate the final model on unseen data, and demonstrate deployment with Azure Machine Learning.

2. Methodology

2.1 Data Description

The dataset contains 50,000 records, seven input attributes, and one binary target. Exactly 5,112 records were labeled as requiring review, representing 10.22% of the dataset; 44,888 did not require review.

Feature	Role / Description
task_type	Categorical task category.
missing_value_count	Count of missing values.
duplicate_flag	Binary duplicate indicator.
format_error_flag	Binary formatting-error indicator.
response_length	Response length.

review_score	Numeric quality/review score.
latency_seconds	Processing latency in seconds.
needs_review	Binary target indicating manual review.

The dataset is synthetic and should therefore be interpreted as a controlled demonstration of an analytical and machine learning workflow rather than evidence of production performance.

2.2 Data Preprocessing

The target was separated from the predictors and categorical and numerical features were incorporated into the classification workflow. The data was divided into 32,000 training records, 8,000 validation records, and 10,000 test records. The test set was kept untouched until final evaluation to reduce the risk of evaluation leakage.

Class imbalance was treated as a primary modeling concern because the review class represented only 10.22% of observations.

2.3 Model Choices

Logistic Regression was chosen because it is suitable for binary classification, efficient, and comparatively interpretable. The project emphasized recall, precision, and F1 in addition to accuracy because a false negative represents a potentially poor-quality record that the system failed to flag.

The evaluated configurations were baseline Logistic Regression, class-weighted Logistic Regression, threshold-tuned Logistic Regression, and hyperparameter-tuned Logistic Regression.

2.4 Training and Validation

The project used Python and scikit-learn. GridSearchCV evaluated C values of 0.01, 0.1, 1, 10, and 100 through cross-validation. The selected C value was 100, with a best cross-validation F1 score of 0.3344. The final decision threshold was 0.70.

3. Results

Model	Accuracy	Precision	Recall	F1 Score
Baseline Logistic Regression	90.23%	63.80%	10.18%	17.55%
Class-Weighted Logistic Regression	71.99%	21.63%	66.34%	32.62%
Threshold-Tuned Logistic Regression	86.15%	33.95%	37.48%	35.63%
Hyperparameter-Tuned Logistic Regression	86.15%	33.95%	37.48%	35.63%

The baseline model's high accuracy was accompanied by very low recall, showing that overall correctness did not reflect the ability to find review-worthy records. The class-weighted model substantially improved recall but generated more false positives. The threshold-tuned and hyperparameter-tuned approaches provided a more balanced result.

3.1 Final Test Confusion Matrix

	Predicted 0	Predicted 1
Actual 0	8,233	745
Actual 1	639	383

The final model correctly identified 383 of the review-worthy test records and missed 639. It correctly identified 8,233 non-review records and incorrectly flagged 745. These counts correspond to 86.16% accuracy, 33.95% precision, 37.48% recall, and 35.63% F1.

4. Discussion

The findings were consistent with the expectation that accuracy alone would be misleading for this imbalanced classification problem. The baseline achieved 90.23% accuracy while detecting only 10.18% of the positive cases. This justified the use of class weighting and threshold analysis.

Class weighting demonstrated the trade-off between recall and precision. The final 0.70 threshold was selected to produce a more balanced operating point. The results also show that the system should be treated as a prioritization tool rather than a replacement for human review.

A deployment challenge occurred when an earlier Azure ML environment created an incompatible NumPy/pandas combination. The problem was resolved by pinning NumPy to 1.26.4 and creating a fresh inference environment with the required package versions.

5. Azure Machine Learning Deployment

The trained model was saved as `data_quality_agent_model.pkl` and registered as `data-quality-agent-model` version 1. The `score.py` inference script loads the model, parses JSON input, generates review probabilities, applies the 0.70 threshold, and returns `needs_review`, `review_probability`, and a human-readable decision.

The model was successfully deployed to the managed online endpoint `data-quality-agent-endpoint` using one `Standard_DS2_v2` instance and `data-quality-agent-env-v3`. A live endpoint test using a high-risk sample returned a 0.993 probability and REVIEW REQUIRED.

6. System Workflow

- Structured input record enters the inference workflow.
- The model processes the seven predictive features.
- Logistic Regression estimates the probability of manual review.
- The 0.70 threshold converts the probability into a binary decision.
- The Azure ML endpoint returns the prediction, probability, and decision.

7. Limitations

- The dataset is synthetic and may not represent real production distributions.
- The final recall of 37.48% means many review-worthy records were missed.
- The 33.95% precision means flagged records would still require verification.

- Large-scale throughput and latency were not formally stress-tested.
- The project does not establish that Logistic Regression is the best possible production algorithm.
- Complete production monitoring and automated retraining were not implemented.

8. Future Improvements

Future development should evaluate additional classifiers, feature engineering strategies, and threshold-selection methods based on explicit business costs for false positives and false negatives. Real production data should be collected and used for external validation. A production system should also include monitoring, data-drift detection, scheduled retraining, access controls, and formal load testing.

9. Conclusion

The Data Quality Agent demonstrates a complete machine learning lifecycle for automated data-quality screening. The project moved from a structured classification problem through preprocessing, model comparison, class-imbalance handling, threshold optimization, final testing, model registration, and cloud deployment.

The final model achieved 86.16% accuracy, 33.95% precision, 37.48% recall, and 35.63% F1 on the untouched test set. The project demonstrates why evaluation metrics must match the actual business objective and provides a practical foundation for future improvements using real data and production monitoring.

Appendix A — Technical Configuration

Component	Configuration
Dataset	50,000 records; 7 predictive attributes; binary target
Positive class	5,112 records / 10.22%
Training / Validation / Test	32,000 / 8,000 / 10,000
Algorithm	Logistic Regression
Grid search C values	0.01, 0.1, 1, 10, 100
Selected C	100
Decision threshold	0.70
Python	3.10.20
scikit-learn	1.7.2
pandas	1.5.3
NumPy	1.26.4
Registered model	data-quality-agent-model:1
Inference environment	data-quality-agent-env-v3:1
Endpoint deployment	Standard DS2 v2, 1 instance

Appendix B — Example Live Prediction

Input: Safety Review; missing_value_count=5; duplicate_flag=1; format_error_flag=1; response_length=90; review_score=60; latency_seconds=150.

Observed output: needs_review=1; review_probability=0.993; decision="REVIEW REQUIRED".